Peterson Magadia

PE02 Linked List and Selection Sort

I expected the linked list to be faster at inserting, since adding an item to the front only means pointing a new node at the old head, which is O(1). The array has to shift every item over one position, which is O(n). When I ran the program, though, the array was faster. I ran it a second time to make sure the result wasn't a fluke, and got the same outcome: the array averaged 8.20 microseconds and the linked list averaged 13.03. I think the main reason is the size of the list. With only 11 items, shifting costs very little, and Python's built-in list is highly optimized. My linked list creates a new node object for every insert, and that overhead outweighed the advantage of not shifting. Delete showed the same pattern, with 2.80 microseconds for the array and 4.31 for the linked list. That result made more sense to me, because both structures have to search for "milk" before removing it, so both are O(n). Lookup was the one operation where the linked list performed better, at 1.05 microseconds compared with 1.99. That's only because I added a tail pointer. Without it, reaching the last item would require walking the entire list. The tradeoff is that the tail has to be updated correctly whenever an item is deleted, which adds complexity and another place for bugs. I also noticed the times varied slightly between runs, so I wouldn't put much weight on small differences. With a much larger list, I would expect the array's shifting cost to become noticeable and the linked list to pull ahead on inserts. I also completed the challenge question, so the linked list can now be sorted using selection sort.